AI-Training Collective Communication in gem5 Garnet

Router-Level Multicast, Express-Link Bypass, and Tensor AllReduce

Haru-LCY and Shealligh

August 10, 2026

Abstract

Modern distributed GPU training repeatedly moves tensors among devices through collectives such as broadcast and all-reduce. Replicating the same data as independent unicast packets wastes shared links, while long multi-hop routes increase synchronization latency. This project explores whether a collective-aware interconnection network can move AI-training data more efficiently. In gem5 Garnet, we implement router-level tree multicast to share packet prefixes, physical express-link bypass to shorten selected routes, and streaming in-network tensor all-reduce to merge contributions before one result broadcast. A trace compiler connects these mechanisms to an executed eight-H100 NCCL microbenchmark. All 10,164 runs in the multicast, bypass, interaction, and bypass-optimization matrices pass completion and statistic checks. Tree multicast reduces internal-link flit traffic by 39.51% on average over 162 pairs and has a 3.588× median completion-latency speedup, although 10 cases regress in throughput. For data traffic on distance-scaled diagonal links, congestion-aware bypass achieves 1.080× geometric-mean average-latency speedup and 1.086× p95-latency speedup over Mesh across 384 paired cases; static bypass achieves 1.068× and 1.072×. Tensor streaming reduces the all-reduce trace window from 80,638,000 to 17,054,500 ticks (4.728× versus scalar-lane replay). These are validated NoC mechanism results, not an end-to-end GPU-training speedup claim. The bypass effect is workload dependent: speedup reaches 1.213× for transpose traffic and 1.266× at offered load 0.7, but is only 1.006× at load 0.05.

1 Introduction

Modern AI models are commonly trained across multiple GPUs. Data parallelism requires gradients to be reduced across workers, while model and tensor parallelism also exchange parameters, activations, and partial results. These communications are organized as collectives—especially broadcast and all-reduce—and often lie on the synchronization path of each training step. As GPU computation becomes faster, moving the same tensor through the interconnect can become a first-order cost.

The conventional network used as our baseline is unaware of this collective structure. A broadcast to N destinations becomes N independent unicast packets, so common path segments carry duplicate flits. Deterministic XY routing also forces every packet through ordinary Mesh links even when a carefully placed shortcut could remove several hops. Finally, decomposing an all-reduce tensor into thousands of independent scalar operations discards the streaming and aggregation opportunities present in the original tensor.

Our project takes inspiration from the collective-aware data movement used in modern multi-GPU systems and asks how much of that efficiency can be exposed inside Garnet. The central idea is to preserve the semantics of an AI collective while reducing redundant packets and hops:

- **Tree multicast** injects one logical packet and replicates it only where destination paths branch, sharing common route prefixes.
- **Physical bypass links** give selected packets an express hop across intermediate Routers, subject to routing safety and wire cost.
- **Tensor all-reduce** keeps a tensor as a multi-flit collective, merges eight rank contributions in the network, and broadcasts one completed result.

A trace compiler and replay path then translate broadcast and all-reduce events captured from a real eight-H100 NCCL microbenchmark into these network operations. The evaluation asks three corresponding questions: whether multicast reduces duplicated network traffic, when bypass latency gains survive a distance-scaled wire model, and whether tensor streaming avoids the cost of scalar serialization.

For Lab 4, the primary submission is **Topic 3: Microarchitecture**: it implements both required technologies, Multicast and Bypass. Trace replay is additional **Topic 4** work, and tensor all-reduce strengthens the common AI-training story. The scope is specifically *AI-training collective communication.* We do not simulate GPU kernels, model layers, optimizer steps, compute/communication overlap, NVLink electrical behavior, or training accuracy; consequently, the results quantify network mechanisms rather than end-to-end training acceleration.

2 Architecture and Implementation

2.1 Baseline and completion contract

The baseline is a Garnet Mesh using deterministic XY routing. For a logical multicast to N destinations, `naive_unicast` creates one standard physical packet for each remote destination. These packets traverse the normal input VC, route computation, VC allocation, switch allocation, crossbar, output VC, and credit pipeline.

A logical operation completes only after every expected destination receives exactly one complete packet with the correct collective ID, round/lane ID, and value. Missing, duplicate, unexpected, wrong-round, wrong-lane, or wrong-value delivery is fatal. Reaching a tick limit does not count as completion.

2.2 Router-level tree multicast

In `tree_multicast`, the Network Interface injects one packet whose flits carry a 64-bit destination bitmap. A Router prunes destinations already served, computes the union of required output branches, produces a local copy only when selected, and keeps branch state across head/body/tail flits. Fanout is atomic: a flit advances only when every selected branch has an output VC/credit. This makes multi-flit delivery safe under backpressure but can make a free branch wait for a blocked branch. The bitmap limits one multicast domain to 64 Routers.

The multicast datapath is a dedicated Router path. It performs its own output-VC and credit checks before placing copies into output units; it does not traverse exactly the same switch-allocation/crossbar path as baseline unicast. Consequently, internal-link traffic reduction is the cleanest structural comparison. Latency speedup measures the complete implemented mechanism, including this fast path, and must not be attributed only to tree sharing.

2.3 Physical bypass links and routing

`Mesh_Bypass` adds real bidirectional Garnet `IntLinks`. Diagonal and stride placement families are evaluated. A generated all-pairs oracle chooses at most one source express hop followed by deterministic XY; its channel-dependency graph is checked to be a DAG. Two link models bound the timing assumption:

- **optimistic**: a shortcut is a one-cycle upper bound;
- **distance-scaled**: latency/serialization reflects Manhattan wire span.

The cost record reports extra links, total wire-length proxy, maximum Router radix, added input ports, and buffer-slot proxy.

The optimized policy adds bounded congestion lookahead. For an unordered data VNet, the source compares free downstream VCs on the oracle's express port and the ordinary XY first port. The shortcut's reverse credit path also models a coarse occupancy signal for the landing Router's next output. A busy landing port causes XY fallback; when it is idle, source express-versus-XY free-VC pressure and packet length make the final choice. A compact epoch monitor updates one hot-destination bit per Router every $16N$ injected data packets; a destination exceeding four times the uniform share is marked skewed. One-flit packets fall back to XY for such many-to-one destinations and otherwise require a two-VC express advantage. Four-flit packets may use an idle express/XY tie to capture the pipeline-hop saving, while longer packets require the two-VC advantage. Packets longer than 32 flits also use XY because a long wormhole reservation can monopolize a shortcut. The packet-length cutoff is a command-line parameter. Ordered VNet behavior remains static. The decision occurs only at the source and every suffix is XY. Thus the ordinary XY dependency graph remains acyclic, while express channels have only outgoing dependencies into that graph and cannot close a cycle.

For multicast, a cost-aware policy uses a source express hop only when the destination's ordinary XY path shares no edge with another destination path. This avoids destroying useful tree sharing. A directed 2×2 diagonal case proves that the joint path can use a shortcut; full-group collectives often preserve ordinary paths instead.

Figure 1 summarizes the joint design. The example has three destinations. Replicated unicast would inject three packets; tree mode injects one bitmap packet. The isolated destination at Router 0 can use the diagonal express link, while destinations 3 and 15 retain their shared ordinary prefix and branch only inside the network.

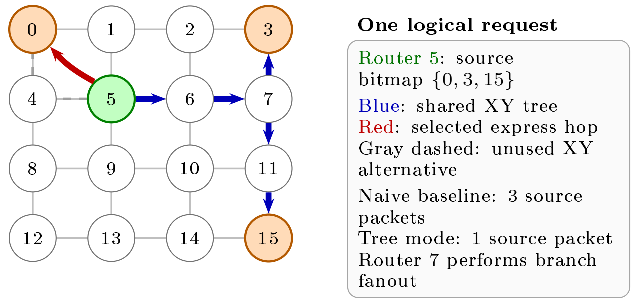

Figure 1: Integrated multicast and bypass example on a 4×4 Mesh. Cost-aware routing uses an express hop for an isolated destination but keeps the shared XY prefix for the other destinations. The diagram shows logical structure; link timing is evaluated separately by the two wire models.

2.4 Trace compiler and tensor all-reduce

The checked-in source trace was captured by executing a controlled collective microbenchmark on eight NVIDIA H100 80GB HBM3 GPUs with PyTorch 2.8.0+cu128, CUDA 12.8, and NCCL 2.27.3. The 48 events include 15 measured broadcasts and 15 measured all-reduces at 1, 4, and 16 MiB. Byte volume and relative release time are explicitly scaled by 1/1024; a flit is 16 bytes.

Broadcast events lower to variable-length tree multicast. The legacy all-reduce lowering serializes each tensor flit as an independent scalar lane. The tensor lowering instead creates 15 multi-flit logical requests, with 64–1,024 flits per rank and one to four chunks per request. Router state is keyed by collective/chunk/lane; after all eight contributions merge to the known sum of 36, the result is tree-broadcast. Backpressure gates vary link latency, Router latency, and outstanding requests. The current accumulator uses an unbounded software map and does not yet model a finite entry table or adder pipeline.

Figure 2 distinguishes the two trace lowerings. Both perform the same total rank contributions and Router flits, but scalar replay turns every tensor flit into a separate logical operation, whereas tensor mode keeps multi-flit requests together through reduction and result broadcast.

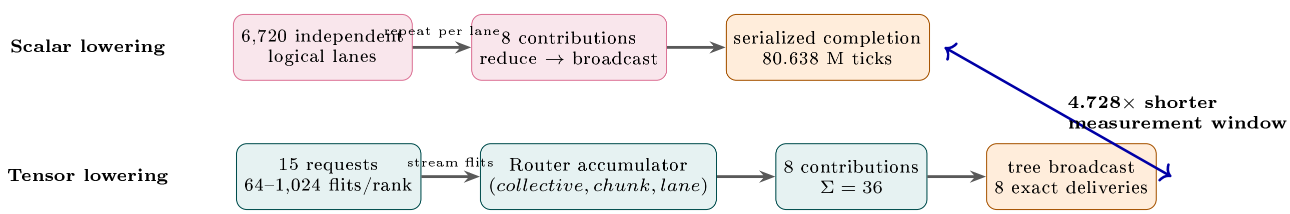

Figure 2: Scalar-lane replay versus the implemented tensor all-reduce path. Tensor mode maintains request/chunk/lane identity while Routers merge eight rank contributions, verify the sum, and multicast the result.

3 Experimental Method

The simulator binary is `build/Garnet_standalone/gem5.opt`, built from gem5 v23.0.0.1 at revision `77fcf26d57`. The full hash is recorded in the manifest. The host exposes 128 logical CPUs but the cgroup quota is 64 CPU equivalents; compilation and large sweeps therefore use at most 64 workers.

The multicast performance matrix uses one 4×4 Mesh and source Router 5; destination counts are 4, 8, and 16; packet lengths are 1, 4, and 16 flits; request probabilities are 0.1, 0.5, and 1.0; background rates are 0 and 0.1; seeds are 1, 7, and 17. Each pair measures 100 requests after 20 warmup requests and before 20 cooldown requests.

G9 uses 4×4 and 8×8 Meshes; uniform-random, transpose, bit-complement, and hotspot traffic; 1, 4, 16, and 64-flit packets; 16 offered loads from 0.01 to 3.0 flits/node/cycle; seeds 1, 7, and 17; and all four placement/wire-model

Table 1: Final quantitative experiment inventory.

Experiment	Raw runs	Pairs/points	Main dimensions
Multicast performance	324	162 pairs	fanout, flits, load, background, seed
Bypass performance (G9)	7,680	6,144 pairs	size, traffic, flits, load, design, seed
Bypass optimization	1,536	768 Mesh pairs	routing mode, traffic, flits, load, seed
Multicast × bypass (G10)	624	416 pairs	mode, wire model, flits, group
Scalar × tensor trace	4	2 pairs	Mesh XY and Mesh Bypass

combinations. There are 1,000 warmup, 100,000 drain, 5,000 measurement, and normally 1,000,000 cooldown cycles. The manifest records 17 cases requiring the longer cooldown window.

G9 is retained as a broad topology/wire-model screening study, but it injected all packets into control VNet 0. That VNet has one buffer per VC and is not a sound proxy for 4–64-flit AI tensor data. The optimization study therefore maps both Mesh and bypass traffic to data VNet 2 (four buffers per VC), and uses only the hardware-relevant distance-scaled diagonal design. It covers the same two sizes, four traffic patterns, four packet lengths, seeds 1, 7, and 17, and offered loads 0.05, 0.1, 0.3, and 0.7. Static and adaptive modes each contain 384 paired seed cases; their offered source/destination sequences are identical.

For paired latency L, internal-link flits F, and throughput Q, we report

$$S_L = L_{base}/L_{new}, \qquad R_F = 1 - F_{new}/F_{base}, \qquad I_Q = Q_{new}/Q_{base} - 1.$$

All arithmetic means, minima, maxima, and negative cases remain in released CSV/JSON files.

4 Correctness Results

The one-command full gate passes all seven stages: 11 legacy collective cases; 208 paired multicast cases; the 30-request H100 broadcast replay; 14 tensor correctness cases; 32 tensor backpressure cases; tensor H100 replay; and paired scalar/tensor replay. The trace tools additionally pass 12 unit tests. The bypass baseline-equivalence gate passes 2×2, 3×3, and 4×4 pairs, comparing 11 statistics in each. G10 passes all 624/624 interaction runs.

During the new G10 run, 40 cases initially failed only the runner's theoretical Router-flit expectation: the checker counted a plain XY tree even when the cost-aware policy correctly selected an express edge. The checker now reconstructs the policy from the independently dumped routing oracle. The simulator binary was not changed; after the test-only correction all 624 runs pass exact delivery, source-flit, Router-flit, physical-packet, and DAG checks.

The current-revision bypass optimization adds 1,536/1,536 passing runs. Its topology dump records whether adaptive admission is enabled, the runner checks that metadata, and both modes use the same 19-file source-hash set. The adaptive choice is exercised by express-link activity changes while packet and flit injection/receipt remain exact.

5 Quantitative Results

5.1 Multicast

Across 162 pairs, mean internal-link flit reduction is 39.51% (range 16.67–53.13%). By destination count it is 26.19%, 39.22%, and 53.13% for 4, 8, and 16 destinations respectively. This monotonic fanout trend is the central evidence that replication is shared inside the network.

Latency speedup has mean 4.790×, median 3.588×, and range 1.200–22.512×; all 162 values exceed one. Mean throughput change is +190.89% and median is +121.66%, but the distribution is skewed by full-group traffic. Ten of 162 cases regress, all with four destinations; the worst is −18.24%. Reduced link traffic therefore does not guarantee higher throughput under atomic fanout.

5.2 Bypass links

The broad mean hides important negative evidence. At offered load at most 0.1, diagonal and stride distance-scaled latency speedups are only 0.962× and 0.936×. A 4×4 diagonal placement adds 9 undirected links, wire proxy 18,

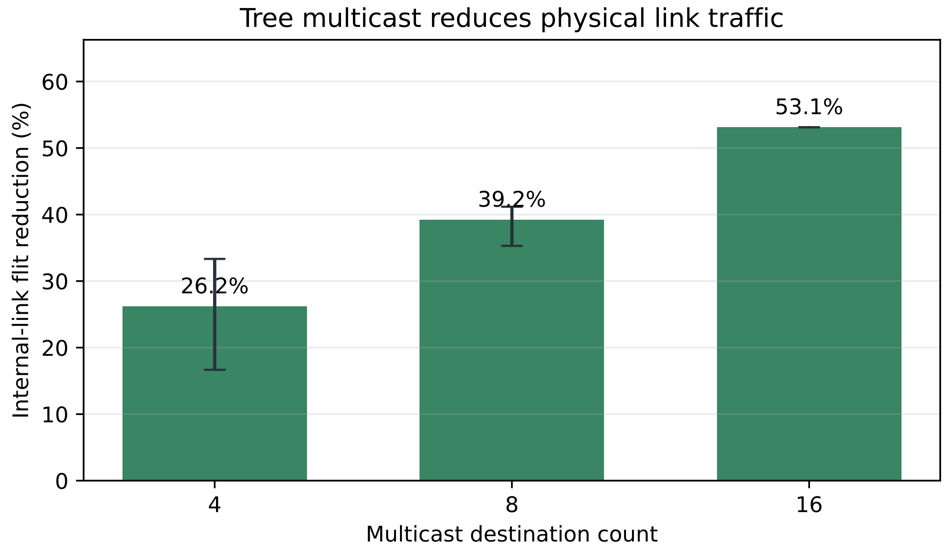

Figure 3: Current-revision multicast internal-link flit reduction. Error bars span observed minima and maxima, not confidence intervals.

Table 2: Mean G9 result. Each design contains 1,536 paired seed cases (512 seed-aggregated points).

Design	Latency speedup	Throughput change	Traversal reduction
Diagonal, distance-scaled	1.629×	+6.41%	−2.82%
Diagonal, optimistic	1.723×	+10.61%	−7.03%
Stride, distance-scaled	0.981×	−1.94%	+15.50%
Stride, optimistic	1.103×	+3.68%	+10.10%

and maximum Router radix 8; on 8×8 it adds 49 links and wire proxy 98. Stride removes more Router traversals but adds 16/96 links and wire proxy 32/192. Hop reduction alone is therefore not a sufficient hardware argument.

The corrected data-VNet A/B study then optimizes the chosen diagonal, distance-scaled design. Because speedups are ratios with a long saturation tail, Table 3 uses geometric means as its primary aggregate. Congestion admission improves geometric-mean average-latency speedup by 1.1% relative to static bypass and p95 by 1.4%; mean Mesh-relative throughput remains positive at +0.83%. More importantly, it prevents the static bit-complement overload cases: the worst Mesh-relative speedup rises from 0.128× to 0.980×, and regressions exceeding 5% fall from 44 to zero of 384 cases.

Table 3: Distance-scaled diagonal bypass on data VNet 2 (384 paired seed cases per mode). Latency columns are geometric-mean Mesh/bypass speedups.

Mode	Avg. latency	p95 latency	Mean throughput	> 5% regressions
Static oracle	1.068×	1.072×	+1.92%	44
Congestion-aware	1.080×	1.086×	+0.83%	0

By traffic pattern, adaptive speedups are 1.022× for uniform-random, 1.213× for transpose, 1.044× for bit-complement, and 1.052× for hotspot. At offered load 0.7 the speedup is 1.266×; at low load 0.05 it is 1.006×. The original 4×4 hotspot/load-0.3 weakness is removed: its 12 cases have 1.091× geometric-mean speedup and a 1.000× minimum. Across the full matrix, eight mild regressions remain, but the worst is 0.980× and none exceeds 5%. Thus the bypass result is significant for structured or heavily loaded NoC traffic, while it is intentionally close to neutral when the network is lightly loaded. The adaptive admission policy is essential: static bypass has a useful mean but can produce pathological individual regressions.

G10 further shows that interaction is workload dependent. Distance-scaled mean latency speedup is 0.965× for replicated unicast and 0.988× for tree multicast; optimistic values are 1.029× and 1.022×. The cost-aware tree uses

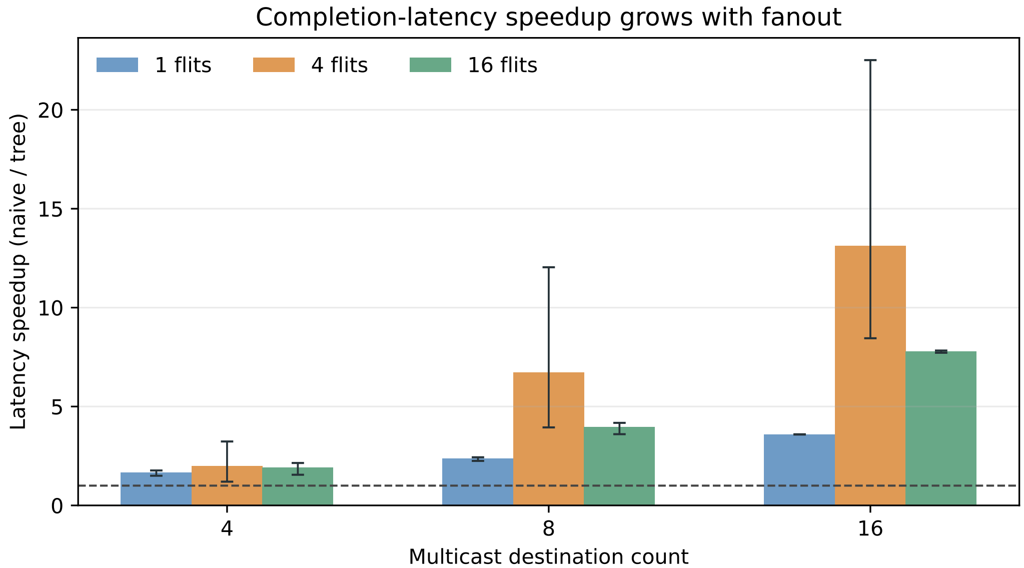

Figure 4: Multicast completion-latency speedup by fanout and packet size.

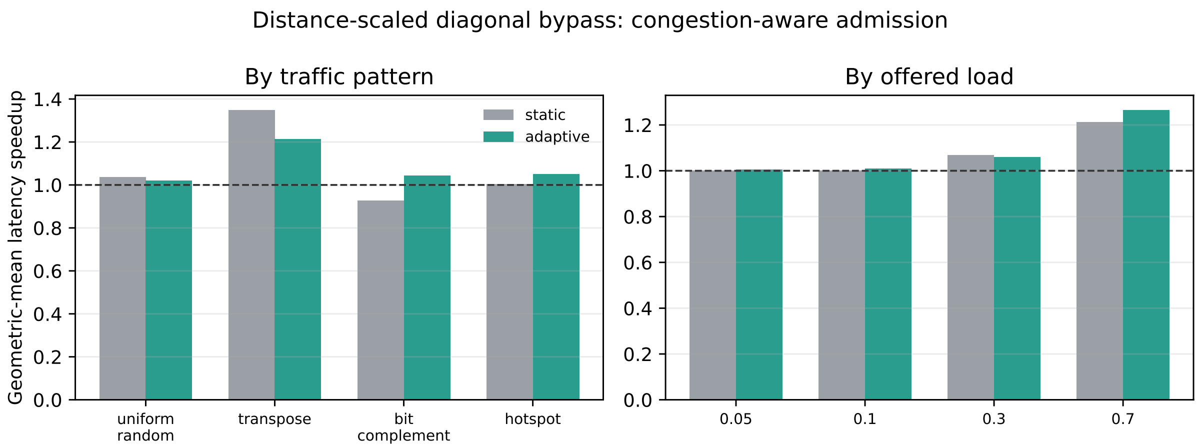

Figure 5: Congestion-aware admission improves the selected physical-wire design across traffic patterns and offered loads while eliminating all regressions larger than 5%.

85,000 express-link flits across the paired runs, versus 357,000 for unicast. It uses shortcuts selectively rather than forcing every multicast branch onto an express link.

5.3 Real-H100 trace and tensor all-reduce

Both Mesh designs complete all 30 broadcast requests, 6,720 source flits, and 210 destination deliveries in 17,047,500 measurement ticks; p95 is 643,500 ticks and wire-flit distance is 47,040. Express usage is zero, so this is a neutral bypass result.

For tensor all-reduce, both designs complete 15 logical requests, 53,760 rank contributions, 53,760 exact reductions, 53,760 deliveries, and 147,840 Router flits. The tensor window is 17,054,500 ticks, p50/p95/p99 logical-request latencies are 647,500/2,567,500/2,567,500 ticks, and peak active lanes are two. Reduce and broadcast each contribute 47,040 internal-link flits.

The scalar window is 80,638,000 ticks, giving 4.728× measurement-window speedup on both Mesh XY and Mesh Bypass. Source and Router flit counts are identical between lowerings. The gain comes from representing a tensor as a multi-flit collective rather than serializing 6,720 independent logical lanes; it is not a measured GPU training speedup. Express usage is again zero for this all-rank convergence/broadcast tree.

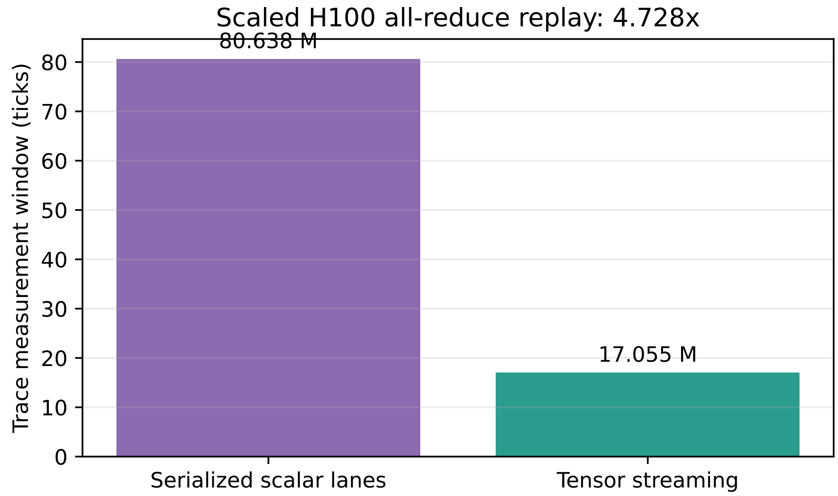

Figure 6: Same scaled H100 all-reduce trace and topology: tensor streaming versus serialized scalar-lane lowering. The p95 values are not compared because their granularity differs.

6 Limitations and Threats to Validity

- Multicast performance uses one 4×4 source placement; corner, edge, and center sources are not separated.
- The multicast fast path does not share the entire normal unicast switch-allocation/crossbar pipeline. Latency results combine tree sharing and microarchitectural-path differences.
- Atomic fanout couples branches, which explains why traffic can fall while throughput regresses.
- Bypass wire cost is a Garnet latency/serialization and static-cost proxy, not area, power, repeater, or timing-closure evidence.
- Bypass lookahead reaches only the express landing Router. Eight of 384 optimized cases remain mildly below Mesh, although none is more than 5% slower. The occupancy and epoch destination-skew signals are modeled without separate propagation-delay, counter-area, or wire-cost accounting.
- The robust policy deliberately sends packets above 32 flits over XY. This removes long-packet shortcut regressions but also gives 64-flit cases no bypass benefit; segmentation is future work.
- The H100 source is an executed collective microbenchmark, not a full model trace. Scaling by 1/1024 changes absolute timing and volume.
- Tensor accumulator capacity and arithmetic latency are not finite hardware resources in the current implementation.
- Results characterize this simulator mechanism; they do not measure NVLink/NVSwitch hardware or end-to-end model throughput.

7 Division of Labor

Repository history gives the following auditable two-person split:

- **Haru-LCY**: multicast baseline and Router tree datapath; multi-flit/backpressure support; bypass topology, routing oracle, cost/performance/interaction gates; H100 trace capture/compiler/replay; integration and report evaluation.
- **Shealligh**: tensor trace contract/compiler/validator; multi-flit in-network tensor reduction and broadcast datapath; link-latency/credit-conservation statistics; tensor correctness, backpressure, H5/H6, and full-gate runners.

- **Joint**: branch integration, regression review, quantitative rerun, limitation analysis, and final report verification.

8 Reproducibility

Raw outputs are under `report/raw-results/head-77fcf26d/` and are intentionally git-ignored; optimization outputs are under `report/raw-results/bypass-optimization/`; compact accepted CSV/JSON snapshots are under `report/data/`. The following commands reproduce the principal evidence:

```
python3 tests/gem5/lab4/run_lab4_full_gate.py \
  --gem5 build/Garnet_standalone/gem5.opt --jobs 64
python3 tests/gem5/lab4/run_multicast_performance.py \
  --gem5 build/Garnet_standalone/gem5.opt --jobs 32
python3 tests/gem5/lab4/run_bypass_performance.py \
  --gem5 build/Garnet_standalone/gem5.opt --inj-vnet 0 \
  --no-bypass-adaptive-routing --jobs 64
python3 tests/gem5/lab4/run_bypass_performance.py \
  --gem5 build/Garnet_standalone/gem5.opt \
  --output report/raw-results/bypass-optimization-v7/static-data-vnet \
  --inj-vnet 2 \
  --families diagonal --wire-models distance_scaled \
  --offered-loads 0.05 0.1 0.3 0.7 \
  --no-bypass-adaptive-routing --jobs 64
python3 tests/gem5/lab4/run_bypass_performance.py \
  --gem5 build/Garnet_standalone/gem5.opt \
  --output report/raw-results/bypass-optimization-v7/adaptive-data-vnet \
  --inj-vnet 2 \
  --families diagonal --wire-models distance_scaled \
  --offered-loads 0.05 0.1 0.3 0.7 \
  --bypass-adaptive-routing --bypass-adaptive-max-packet-flits 32 \
  --jobs 64
python3 tests/gem5/lab4/run_multicast_bypass_matrix.py \
  --gem5 build/Garnet_standalone/gem5.opt --jobs 64
python3 tests/gem5/lab4/run_tensor_allreduce_paired.py \
  --gem5 build/Garnet_standalone/gem5.opt
python3 report/scripts/build_final_report_data.py
```

The compact data builder rejects wrong run counts, raw errors, inconsistent optimization VNet/routing metadata, incomplete multicast measurements, failed full gates, and missing designs before writing summary CSVs and figures. The optimization manifests hash every modified configuration, topology, runner, routing, network, and output-VC source file used by the current binary.

9 Conclusion

Starting from the communication demands of modern multi-GPU training, this project shows three ways to preserve collective structure inside the network: share common broadcast paths, skip selected intermediate Routers, and aggregate tensor contributions before broadcasting the result. It fully satisfies the two-person Topic 3 requirements because both router-level multicast and physical bypass links are implemented, tested, and compared against appropriate Mesh/unicast baselines. Multicast achieves 39.51% mean internal-link traffic reduction, with benefit increasing with fanout, although 10 throughput regressions and the dedicated fast path qualify the latency result. Bypass is meaningfully effective but workload dependent: adaptive admission achieves 1.080× overall, 1.213× on transpose, and 1.266× at high load, removes the 4×4 hotspot weakness, and reduces regressions larger than 5% from 44 to zero. Its near-neutral low-load result and 32-flit cutoff reject a universal speedup claim; these are NoC latency results, not end-to-end GPU-training throughput. Finally, exact replay of broadcast and all-reduce events from a real eight-H100 microbenchmark connects the mechanisms to AI-training communication. The tensor path's 4.728× reduction is evidence that preserving a streaming collective is more efficient than scalar serialization in this simulator, not an end-to-end training-speedup claim.